



A deep reinforcement learning based searching method for source localization

Yong Zhao^{a,1}, Bin Chen^{a,*,1}, XiangHan Wang^{a,1}, Zhengqiu Zhu^{a,1}, Yiduo Wang^a, Guangquan Cheng^a, Rui Wang^a, Rongxiao Wang^a, Ming He^b, Yu Liu^c

^a College of Systems Engineering, National University of Defense Technology, 109 Deyu Road, Kaifu District, Changsha City, Hunan Province, China

^b Command and Control Engineering College, Peoples Liberation Army Engineering University, China

^c School of Automation Science and Engineering, South China University of Technology, China

ARTICLE INFO

Article history:

Received 15 October 2020

Received in revised form 31 October 2021

Accepted 11 December 2021

Available online 23 December 2021

Keywords:

Autonomous searching method

Deep reinforcement learning

DBSCAN algorithm

Entrotaxis algorithm

ABSTRACT

The localization of hazardous sources (e.g. poisonous gas sources) is an important task regarding the security of human society. To find the unknown source in time, various autonomous source searching methods have mushroomed and been employed over the past decade. This paper designs a fresh source searching approach, namely particle clustering-deep Q-network, PC-DQN, which applies the deep reinforcement learning (DRL) techniques as a source searching approach for the first time. Specifically, the search process is formulated as the partially observable Markov decision process, then converted into the Markov decision process based on the belief state (represented by the particle filter). PC-DQN leverages the density-based spatial clustering of applications with noise (DBSCAN) algorithm to extract the feature of belief state, and employ the deep Q-network (DQN) algorithm to find the optimal policy for the source searching task. Through the comparison with two baseline methods (i.e. RANDOM and Entrotaxis algorithm) under various experimental conditions, the viability of our proposed PC-DQN is testified. Results explicitly reveal that the success rate of the PC-DQN maintains at a high level (beyond 99.6%) in all scenarios in this paper, and the mean search step shows evident superiority over baseline methods in most scenarios. Significantly, we also introduce the transfer learning concept to reuse the well-trained Q-network into new scenarios. These findings show important implications of the DRL-based approach as an alternative and more effective source searching approach.

© 2021 Elsevier Inc. All rights reserved.

1. Introduction

The gas leakage accidents could exert serious consequences on ecology and human life [1], while the casualties and property losses can be significantly reduced when the emergency management bodies or environmental protection bodies are dispatched to implement measures in time. The most essential measure for emergency response is to localize the leakage

* Corresponding author.

E-mail addresses: zhaozong15@nudt.edu.cn (Y. Zhao), nudtc9372@gmail.com (B. Chen), wangxianghan13@nudt.edu.cn (X. Wang), zhuzhengqiu12@nudt.edu.cn (Z. Zhu), wangyiduo14@nudt.edu.cn (Y. Wang), cgq299@nudt.edu.cn (G. Cheng), ruiwangnurd@gmail.com (R. Wang), wangrongxiao12@nudt.edu.cn (R. Wang), paper_review@126.com (M. He), aylaur@scut.edu.cn (Y. Liu).

¹ These authors contribute equally to this paper.

source quickly as well as to master the accurate source terms [2]. Therefore, the source localization problem attracted lots of research attention in recent years [3,4].

One of the widely used approaches to solve the source localization problem is the autonomous source searching technique [5–7]. With the recent development of robotics and information sciences, different types of mobile sensors or so-called searchers (e.g. robots or unmanned aerial vehicles (UAV) equipped with several sensors) have been leveraged for source searching. These searchers are guided to maneuver autonomously toward the source based on available sensing cues and prior information and finally approach the source to determine the source terms (e.g. location, release strength). As a major kind of autonomous searching method, the bio-inspired strategy can guide the movement of searchers based on simple reflections, such as the Zigzag algorithm [8] which is inspired by the foraging behavior of beetles and the Silkworm algorithm [6,7,9,10] imitating the search habit of moths. However, those bio-inspired strategies generally have a poor performance in the real world because of the ubiquitous turbulence.

Cognitive strategy [11–13], as another kind of searching method based on the Bayesian framework, has been applied to tackle the problem in turbulent situations in recent years. In the Infotaxis, which is a typical cognitive strategy, the source searching process is formulated as the partially observable Markov decision process (POMDP) [14] and the estimation of the source term is represented as the probability density function (PDF). Researchers [11,15–17] employed the particle filter to replace the grid-based numerical integration [18] in Infotaxis with the attempt of diminishing the computational complexity. Based on the framework of Infotaxis, the Entrotaxis [3], as an alternative cognitive strategy, designed a new reward function considering the maximum entropy sampling principle, and the computational complexity of Entrotaxis is reduced further. However, the cognitive search strategy is still limited by its high computational complexity, especially when the number of samples in the particle filter increases, which hampers it from being applied in the scenarios of actual emergency response.

The source searching problem is a sequential decision-making problem in essence, in which the searcher has to decide to take appropriate actions in each step to help it find the source with minimum cost (e.g. search step). As one of the paradigms and methodologies of machine learning, deep reinforcement learning (DRL) [19] can be used to solve the decision-making problem in which agents learn policy to guide their behavior to maximize the accumulated reward signal or achieve specific goals in their interactions with the environment. Furthermore, the deep reinforcement learning approaches always calculate and make real-time decisions effectively in the application after the sufficient learning process. With well modeling and abundant training, the deep reinforcement learning method might be well-fitted to solve the source searching problem. Thus, we try the first attempt to apply the deep reinforcement learning technique to design an efficient source searching strategy with a superior formulation in turbulent situations in this paper.

To effectively apply the DRL method in the problem of source searching, we still face several issues: (1) *How to transfer the POMDP problem to the Markov decision process (MDP) problem.* The DRL is widely used to solve the MDP problem, however, the source searching, as a typical POMDP problem [14,20–24], is intractable: The agent cannot perceive the complete state of the environment and it must make decisions under uncertainty of the true state. If the environment has discrete and finite states, the belief state can be represented in a tabular form. Then, the POMDP can be solved by approximating the optimal policy [24]. When the state space is continuous and high-dimensional, the POMDP is still difficult to find a solution. (2) *How to model the state, action, and reward in source searching.* In general, reinforcement learning attempts to learn a value function that takes the current state as input, and generates value scores for each possible action as output [25]. However, different definitions of state, action, and reward have significant effects on the solving of real problems. For example, how to extract features of a state is a key issue in the modeling process. (3) *The learning of the value function.* Q-table [26] is usually employed to store value for each state-action pair. It has good performance when the number of states and actions is limited. However, in the source searching problem, the state is continuous and infinite, and the value of all states can hardly be stored in a table. (4) *The Scarcity of training data.* Generally, deep reinforcement learning requires numerous training data to learn the value function. However, in the source searching problem, we can hardly obtain enough data for training in each scenario. Fortunately, the transfer learning technique can be applied to make use of trained Q-network and reduce the required training experiments and data for searching tasks for similar tasks.

In this paper, we will solve the above-mentioned issues and bring the following novel contributions:

- (1) To the best of our knowledge, this is the first attempt at using DRL to solve the source searching problem. Although previous work [27] had employed reinforcement learning, the modeling of the searching problem in it is recognized as rough. In this paper, the source searching problem is interpreted as a searcher required to choose the next direction for sensing considering the already collected data with the attempt to minimize the number of search steps. As a typical POMDP, a Markovian belief state (denoted by the particle filter) allows the source searching problem to be formulated as an MDP where each belief is represented by a state. Then, we use a clustering algorithm called density-based spatial clustering of applications with noise (DBSCAN) [28] to extract the feature of the belief state. Finally, a DRL approach (deep Q-network algorithm, DQN) [29] is applied to train the network and find the optimal action in each state.
- (2) Considering the extremely large state space, we employ a neural network instead of a table as the value function. In the source searching problem, the states are continuous and large. Therefore, the traditional Q-table method is hard to store all values and retrieve them. Instead, we propose to leverage a neural network to replace the table [19], i.e., leveraging DRL to learn Q-network for our source searching problem.

- (3) We introduce the transfer learning concept to make use of the well-trained Q-network and reduce the required training experiments and data for heterogeneous searching tasks in similar source searching missions. In particular, the transfer learning algorithm can reduce the number of training experiments from 500 to 100 while guaranteeing a similar performance in new tasks.
- (4) Results of extensive experiments in various scenarios show the superiority of our proposed PC-DQN algorithm. Compared with RANDOM and Entrotaxis (two baseline algorithms), the PC-DQN algorithm keeps a higher success rate and needs fewer search steps typically.

This paper proposes a novel, promising source searching method called the PC-DQN algorithm, and the paper is organized as follows: In [Section 2](#), the diffusion model of leaked gas is built based on the convection-diffusion model, and the source searching process is first formulated as a POMDP, then transformed into an MDP based on the belief state. In [Section 3](#), the DQN algorithm is introduced and the DBSCAN algorithm is employed to extract features of belief state, then the PC-DQN algorithm is presented. In [Section 4](#), a series of experiments are implemented to demonstrate the performance of the PC-DQN searching method. Moreover, the transfer learning experiments are also carried out in this section. A brief conclusion and future work are given in [Section 5](#).

2. Problem formulation

2.1. The diffusion model of leaked gas

In this paper, the convection-diffusion model [\[18\]](#) based on the Convection-Diffusion Equation is adopted to establish the forward diffusion model of leaked gas. Specifically, a point leakage source located at $l_0 = \{x_0, y_0\}$ in the two-dimensional search domain Z , releases gas particles at an invariable rate Q , and those gas particles diffuse across space with the isotropic diffusivity ζ and stable wind V . Then the mean particles concentration $c(l|l_0)$ at the location $l = \{x, y\} \in Z$ can be formulated as:

$$c(l|l_0) = \frac{Q}{4\pi\zeta|l-l_0|} e^{\frac{-(y-y_0)V}{2\zeta}} e^{\frac{-|l-l_0|}{\lambda}} \quad (1)$$

where $\lambda = \sqrt{\zeta\tau/(1+V^2\tau/4\zeta)}$, and τ is the average lifetime of gas particles. Affected by the turbulence, a spherical sensor of size ψ located at l will randomly encounter with dispersed particles at the mean rate $\rho(l|l_0)$, formulated in [Eq. \(2\)](#):

$$\rho(l|l_0) = 4\pi\zeta\psi c(l|l_0) \quad (2)$$

The Poisson random process is employed to model the random contact between the sensor and particles; therefore, the probability of the sensor encounters with particles d times per unit time is presented as:

$$P(d(l)|l_0) = \frac{\rho(l|l_0)^d}{d!} e^{-\rho(l|l_0)} \quad (3)$$

An illustrative diagram of the convection-diffusion model is shown in [Fig. 1](#), and [Fig. 1 \(a\)](#) shows the mean rate of an encounter between the sensor and particles, while [Fig. 1 \(b\)](#) shows the actual encounter times with random effect. Comparing the two pictures in [Fig. 1](#), the model can simulate the diffusion feature, which is intermittent and sporadic, in turbulence with high computational efficiency.

2.2. Modeling the searching process as the belief MDP

In the process of source searching, the searcher should approach the leakage source within minimum steps, and obtain an accurate estimation of source term parameters. Since the searcher does not know the actual parameters, it can only extract the information related to the source terms from sensor measurements, and make decisions about the next move based on the extracted information. This process can be modeled as the POMDP [\[24\]](#), which consists of seven elements as follows:

- S is the set of states,
- A is the set of actions,
- T is the conditional transition probabilities between states,
- R is the reward function,
- Ω is the set of observations,
- O is the conditional observation probabilities,
- γ is the discount factor.

The POMDP formulates a sequential process: at each step, the environment is in some state $s \in S$ that the agent (the searcher) does not know. The agent takes an action $a \in A$ to interact with the environment, correspondingly, the

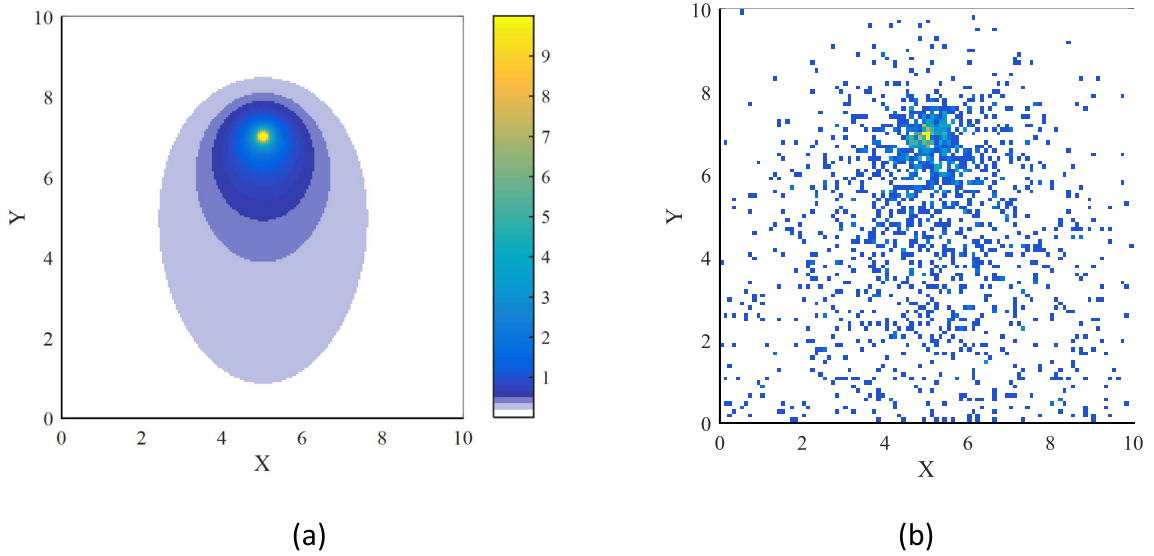


Fig. 1. An illustrative diagram of the Convection-Diffusion model. (a) Exhibits the mean rate of the encounter between the sensor and particles, and (b) represents the actual encounter times with random effect.

environment transfers to another state s' with probability $T(s'|s, a)$. At the same time, the agent receives an observation $o \in \Omega$ depending on the new state s' and the taken action a with probability $O(o|s', a)$. Subsequently, the agent obtains a reward $r = R(s, a)$, which is a numerical evaluation of the just event. Then the process repeats until specific termination conditions are triggered, and finally, the agent obtains the accumulative reward $\sum_{k=0}^{\infty} \gamma^k r_k$, where r_k is the reward earned at time step k . The discount factor γ determines how much immediate reward is favored over more distant rewards. The goal is for the agent to adjust the policy according to the accumulative reward so as to learn an optimal policy, which can guide the agent to choose appropriate action at each step that maximizes the expected accumulative reward $E[\sum_{t=0}^{\infty} \gamma^t r_t]$.

In the source searching problem discussed in this paper, the state s is the actual parameters of the leakage source term, which is assumed as invariable during the searching process. The action set A contains four optional actions that control a searcher move forward, backward, left, and right $\{\uparrow, \downarrow, \leftarrow, \rightarrow\}$ with fixed step size respectively. The observation o is the encounter time between sensor and gas particles and the conditional observation probabilities can be calculated by Eq. (3).

The agent cannot directly observe the entire state of the environment, but it can estimate the true state based on current and historical observations. The estimation of the true state is called belief state b , and the $b(s)$ denotes the probability that the environment is in state s . Given the $b(s)$, after taking action a and observing o , the belief state is updated as:

$$b(s') = \eta O(o|s', a) \int T(s'|s, a) b(s) ds \quad (4)$$

where η is a normalizing constant calculated as:

$$\eta = \int O(o|s', a) \int T(s'|s, a) b(s) ds ds' \quad (5)$$

In the source searching process, since the actual parameters are invariable, which means $s' = s$ and $T = 1$. Therefore, Eq. (4) can be simplified as:

$$b'(s) = \frac{O(o|s, a) b(s)}{\int O(o|s, a) b(s) ds} \quad (6)$$

If the belief state is recognized as the state in MDP, the POMDP can be transformed into the belief MDP. The optimal policy of the belief MDP usually includes the actions for information collection, which can improve the accuracy of the estimation of the actual state, thus allowing the agent to make better decisions in the future.

2.3. Belief state iterative solution based on the particle filter

Since the belief state space B is continuous and there are infinite belief states in B , the update of the belief state shown in Eq. (6) is difficult to be implemented through numerical calculations. The traditional method divides the searching area into a set of cells by a grid [18]. The center of the cell is considered as the potential location of the gas source. In this way, the

belief state space B is transferred to be discrete and finite, which can be estimated by various algorithms. However, this approach has some limitations. As the release source may locate at the edge between two or four cells, it is hard to estimate and search for an accurate source position according to the information of center positions of all cells.

Alternatively, the particle filter [30] is adopted in this paper to carry out the iterative calculations of the belief state for its flexibility and accuracy. The belief state $b_k(s)$ in time step k is approximated by a set of N random samples $\{(p_k^i, w_k^i)\}_{i=1:N}$, where p_k^i denotes a point estimation regarding source parameters and w_k^i is the associated weight ($\sum_{i=1}^N w_k^i = 1$). After sampling, the belief state can be presented as:

$$b_k(s) = \sum_{i=1}^N w_k^i \delta(p - p_k^i) \quad (7)$$

where the $\delta(\cdot)$ is the Dirac delta function. Sequential importance sampling is employed, and it is assumed that there is a proposal distribution $q(s)$, which is similar to the belief state $b_k(s)$. Then the sampling will be extracted over the proposal distribution $q(s)$, and the particle filter will be updated as:

$$\begin{cases} \bar{w}_k^i = w_{k-1}^i \frac{P(p_k^i | p_{k-1}^i) O(o | p_k^i, a)}{q(p_k^i | p_{k-1}^i, o)} \\ w_k^i = \bar{w}_k^i / \sum_{i=1}^N \bar{w}_k^i \end{cases} \quad (8)$$

Because of the invariable source parameters, the point estimation $p_k^i = p_{k-1}^i$ and the $q(s)$ can be proposed as $q(s) = b_{k-1}(s)$. Then Eq. (8) can be simplified as:

$$\begin{cases} \bar{w}_k^i = \bar{w}_{k-1}^i O(o | p_k^i, a) \\ w_k^i = \bar{w}_k^i / \sum_{i=1}^N \bar{w}_k^i \end{cases} \quad (9)$$

To ensure the performance of the particle filter, the resampling step is carried out when the number of effective samples falls below a threshold, and the Markov Chain Monte Carlo (MCMC) move step is also added to improve the diversity of the samples.

3. The solution based on the PC-DQN algorithm

This section mainly introduces the PC-DQN algorithm. Firstly, the DQN algorithm is briefly introduced which combines the Q-learning algorithm with the neural network. It adopts the experience replay mechanism and target network to enhance the training stability and convergence. Then, the density-based clustering algorithm DBSCAN is employed to extract the feature of the belief state, so the feature can be leveraged as the input of the neural network in the PC-DQN. Finally, we use a running case to illustrate how the PC-DQN algorithm solves the source searching problem.

3.1. The DQN algorithm

The traditional tabular Q-learning [26] method stores the value functions through tables when the environment has finite states. However, when faced with the problem of continuous state space or action space, it is necessary to design an appropriate value function approximator to replace the table. The DQN algorithm leverages the neural network as the value function approximator by taking the state feature as the input of the neural network and the state-action value as the output [19].

Before the emergence of the DQN, neural networks have been employed to approximate the value function in reinforcement learning, but the learning process would be unstable or even not convergent. To solve this problem, the DQN designs two significant mechanisms: experience reply and the target network [29]. The learning process of DQN is shown in Fig. 2.

The experience $e_t = (s_t, a_t, r_t, s_{t+1})$ of each step will be stored in the memory set $D_t = \{e_1, \dots, e_t\}$. During learning, small batches of experience samples are randomly extracted from the memory set, and parameters of the network are updated using the stochastic gradient descent algorithm. The experience replay mechanism removes correlations in the observation sequence and smooths over changes in the data distribution. In addition to setting up the Q-network θ to estimate the value function, the DQN also employs a target network θ^- to calculate the Q target value separately, which is only periodically updated with the parameters in network θ . The parameters of the network θ are updated through the following formula:

$$\theta_{t+1} = \theta_t + \alpha(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)) \nabla Q(s, a; \theta) \quad (10)$$

where the α is the learning rate.

The structure of the Q-network in this paper is shown in Fig. 3, which is designed and verified via several pre-experiments. There is only one hidden layer with 10 nodes, and the hidden layer is fully connected with the output layer

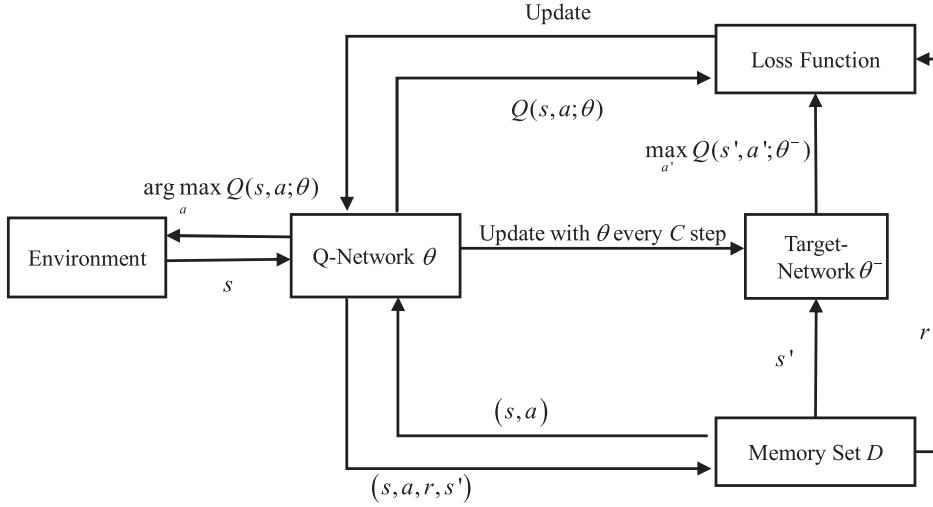


Fig. 2. The learning process of the DQN.

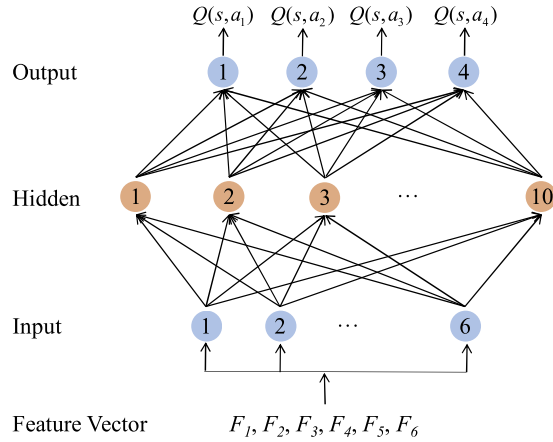


Fig. 3. The structure of the Q-network.

and input layer. The activation of each layer is ReLU. The input of the Q-network is the hand-crafted feature vector of the belief state and the output is the value of the state-action pair.

3.2. The design of feature of belief state

The belief state is denoted by the particle filter that consists of a set of samples. Distributing all samples on the two-dimensional search domain according to their estimation of source location, the probability map of the source is presented clearly, as shown in Fig. 4 (a). The density of samples in a region can reflect the possibility of the source position falling into this region. Therefore, the feature of the belief state can be extracted based on a density-based clustering algorithm, namely DBSCAN [28]. It can divide a set of samples with high enough density into a cluster and discover clusters with arbitrary shapes in the noisy dataset. The key idea is that for each sample of a cluster, the neighborhood of a given radius must contain at least a minimum number of samples. Therefore, the two parameters in the DBSCAN algorithm are defined as *Eps* and *MinPts*, represented by the given radius and a minimum number of points respectively.

As shown in Fig. 4 (b), using the DBSCAN algorithm, most samples are categorized into a cluster and the residual samples that do not belong to any cluster are noise samples. The number of samples in a cluster indicates the possibility of the source position falling into this cluster. However, some clusters contain few samples, and it brings less information. Therefore, if the number of samples in a cluster exceeds the proportion ξ of the total number of samples, the cluster is defined as a valid cluster in this paper. The cluster that contains the most samples among all clusters is defined as the maximum cluster. The

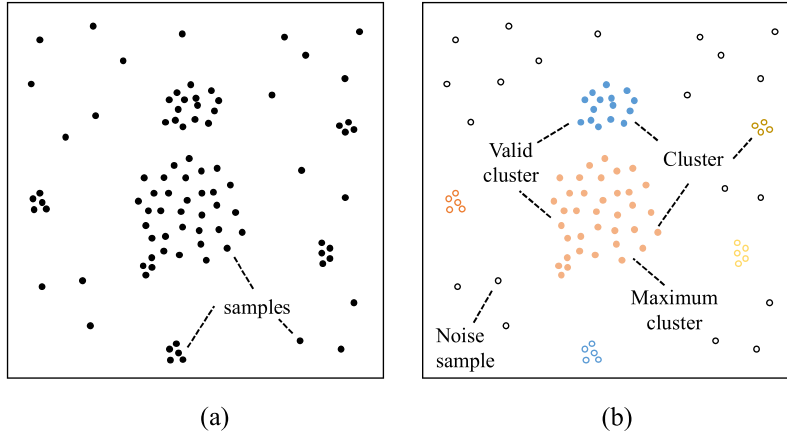


Fig. 4. The distribution of samples is shown in (a) and the clustering information extracted by the DBSCAN algorithm is shown in (b). The black points in (a) denote samples. In (b), clusters are recognized with color, and black hollow points denote noise samples. The clusters with solid points are valid cluster since the number of samples in them exceed the threshold. The cluster with brown points is the maximum cluster because it contains the most samples.

$l_m = (x_m, y_m)$ is the mean point of the maximum cluster, and the $l = \{x_s, y_s\}$ is the position of the searcher. Then, the features $\phi = \{F_1, F_2, F_3, F_4, F_5, F_6\}$ of the belief state are defined as:

- F_1 : the ratio of noise samples to total samples
- F_2 : the ratio of the samples in the maximum cluster to total samples
- F_3 : the number of valid clusters
- F_4 : $(x_m - x_s)/X$
- F_5 : $(y_m - y_s)/Y$
- F_6 : $|F_4| - |F_5|$

where $\{X, Y\}$ denotes the size of the search domain. These six features describe the belief state from different perspectives. F_1 , F_2 , and F_3 are related to the distribution of samples, and they reflect the convergence degree of samples. F_4 , F_5 , and F_6 are relevant to the distance between the current position of the searcher and the most possible source location since the maximum cluster contains most samples. F_4 and F_5 are divided by the size of the search domain, so these features are extensible in many search domains with different sizes.

3.3. The framework of the PC-DQN

Based on the models and methods discussed before, we clarify the state, action, and reward in the source searching problem and give the framework of the PC-DQN.

State: The state is represented by a set of samples based on the particle filter and the features of the state are extracted by the DBSCAN algorithm.

Action: The action set A contains four entries that maneuver the searcher to move forward, backward, left, and right with fixed step size, respectively. This action set is widely used as a baseline in previous works [11,12,31]

Reward: Since the goal is to minimize the search steps, the searcher obtains a negative reward $R_n = -1$ in each step. When the source has been found, the searcher can get a positive reward $R_p = +20$.

The PC-DQN source searching algorithm is shown in Algorithm 1 and its framework is presented in Fig. 5. The input contains initial parameters, such as scenario size, release strength, and wind speed. The output is the trained network. In each episode, the scenario is initialized randomly including the particle filter, source position, and searcher's position.

In each cycle of the episode, the searcher obtains the observation o_{t+1} and the reward r_t at the new position. Then, the new observation is utilized to update the belief state s_{t+1} . In step 6 in Algorithm 1, the feature $\phi(s_{t+1})$ of the belief state is extracted by the DBSCAN algorithm. Inputting the features to the Q-network, then the value function of each optional action is acquired. The searcher selects action a_{t+1} based on the ϵ -greedy strategy. For example, the searcher chooses the action $\{\uparrow\}$ in Fig. 5. In step 11 in Algorithm 1, the searcher executes an action a_{t+1} . Finally, the network is updated as shown in steps 13–19 in Algorithm 1. According to the mechanism of experience reply, we store the transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in the memory set D . Then a new transition $(\phi_j, a_j, r_j, \phi_{j+1})$ is sampled uniformly at random from D . In step 14, if s_{t+1} is a state of termination, the total reward ω_t is set to r_t ; otherwise the total reward is calculated by the target network θ^- . In the end, a gradient descent is performed to update the parameters of approximation network θ . Note that the target network θ^- is reset every C steps and

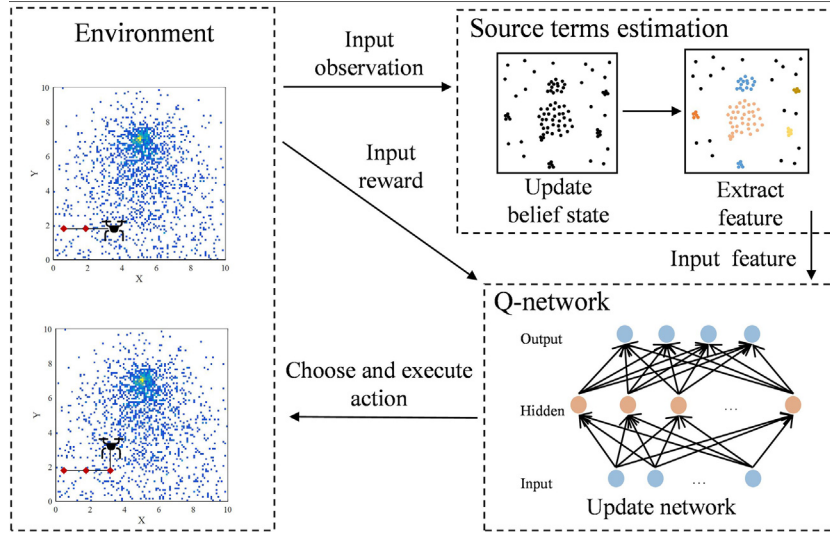


Fig. 5. The framework of the PC-DQN algorithm. The searcher obtains observation and reward from the environment, and the observation is used to update the belief state. Utilizing the DBSCAN algorithm to extract features of belief state and the features are inputted into Q-network to obtain the chosen action. Besides, the Q-network is updated based on observation and reward.

the algorithm will shift to a new cycle in case the source is found or the search steps exceed the preset threshold (terminal condition).

Algorithm 1: PC-DQN

Input: Initialized parameters

Output: Trained network

```

1  For episode = 1:  $M$  do
2    Initialize the scenario
3    For  $t = 1: T$  do
4      Obtain the observation  $o_{t+1}$  and reward  $r_t$ 
5      Update the particle filter and get the belief state  $s_{t+1}$ 
6      Utilize the DBSCAN algorithm to extract the feature  $\phi_{t+1} = \phi(s_{t+1})$ 
7      If random value  $< \epsilon$ , then
8        select a random action  $a_{t+1}$ 
9      Else
10       select  $a_{t+1} = \underset{a}{\operatorname{argmax}} Q(\phi(s_{t+1}), a; \theta)$ 
11       Execute action  $a_{t+1}$ 
12       Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $D$ 
13       Sample random minibatch of transition  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $D$ 
14       If the episode terminates at step  $t + 1$ , then
15          $\omega_j = r_j$ 
16       Else
17          $\omega_j = r_j + \gamma \underset{a'}{\operatorname{argmax}} \hat{Q}(\phi_{j+1}, a'; \theta^-)$ 
18       Perform a gradient descent with respect to  $\theta$ 
19       Every  $C$  steps reset  $\theta^- = \theta$ 
20       If satisfy the terminal condition: break
21     End
22   End

```

4. The simulation experiment

In this section, the Q-network is firstly trained in the simulation scenario. Then, a series of source searching experiments with various environmental parameters are conducted on our proposed PC-DQN algorithm as well as the baselines. Finally, to reuse the well-trained Q-network and further reduce the number of training experiments and training data in new scenarios, we introduce the transfer learning technique into our problem.

4.1. Training of PC-DQN

First, a typical simulated scenario is built. It is assumed that there is a gas leakage source in a rectangular area with a side length of 8. The release strength $Q = 1$ and wind speed $V = 0$. These two parameters have a significant impact on the search performance, thus they are discussed via various experiments later. The diffusion parameters are consistent with previous works [3,11]: the diffusion rate $\zeta = 1$, the average lifetime of gas particles $\tau = 250$, and the sensor size $\psi = 1$. The number of samples in the particle filter is set as 3000. The searcher moves at a fixed step size of 0.5, and the source location is required to be estimated without prior information. Noting that all physical quantities above are in arbitrary units (a.u.) for general representativeness [11].

The PC-DQN is set to learn 500 episodes and the hyperparameters are shown in Table 1. In each episode, the source location and the starting point of the searcher are generated randomly. **An episode will be terminated if the search step exceeds 1000, or the source is localized successfully.** Considered the searcher is always equipped with some dedicated sensors (such as vision sensors) to help it to complete the source declaration, thus the source can be found when the distance between searcher and source is less than a step size.

The typical convergence procedure is shown in Fig. 6, which is measured by the average reward in 10 episodes. At the initial stage of the training process, the reward fluctuates dramatically and keeps at a low level, indicating that the PC-DQN has not obtained an effective policy. After hundreds of training episodes, the reward increases gradually with a decrescendo fluctuation, marking the convergence of the training process. Noteworthily, the reward still has a slight fluctuation at the end of training since the reward is related to the number of search steps that is affected by the random distance between the source and the initial position of the searcher.

Table 1
The PC-DQN hyperparameters.

Hyperparameter	Value	Description
minibatch size	32	Number of training samples per update in stochastic gradient descent
replay memory size	100,000	Size of the experience replay dataset
target network update frequency	500	Frequency of updating the target network
learning rate	0.00025	Learning rate α
initial exploration	1	Initial ε value in ε greedy exploration policy
ε greedy	0.0001	Rate at which ε decreases
discount factor	0.99	Discount factor γ

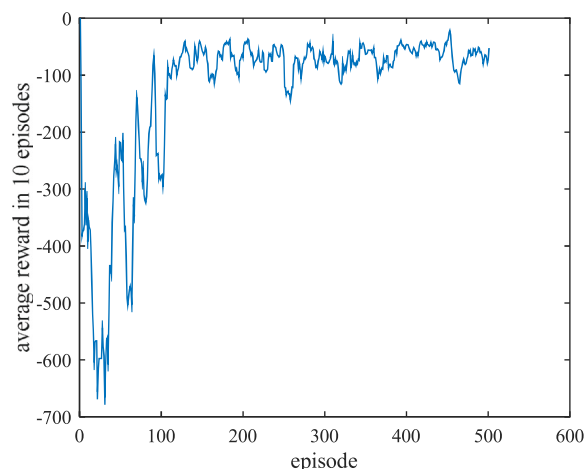


Fig. 6. The typical convergence procedure of the PC-DQN.

4.2. Typical search pattern

A typical source searching process is shown in Fig. 7 and the (a), (b), (c), (d) demonstrate the sequential process at different time steps. The green dots in the figure represent samples of the particle filter, which are also the estimation of the source terms. The red circle denotes the leakage source. The blue solid circle and blue hollow circle represent the starting position and current position of the searcher respectively. The black line denotes the trajectory of the searcher and the points on the trajectory represent that the searcher has obtained measurements at them. The red '+' symbol represents the mean point of the maximum cluster.

At the beginning of the searching process, the searcher does not have enough information about source terms, so the samples are randomly distributed throughout the whole search domain, and the mean point of the maximum cluster is just at the center of the scene. When the searcher has accumulated enough information, samples will gather at certain positions in the scene and the mean point will move accordingly. The searcher tends to move towards the mean point and finally moves to the vicinity of the source at time step $k = 32$.

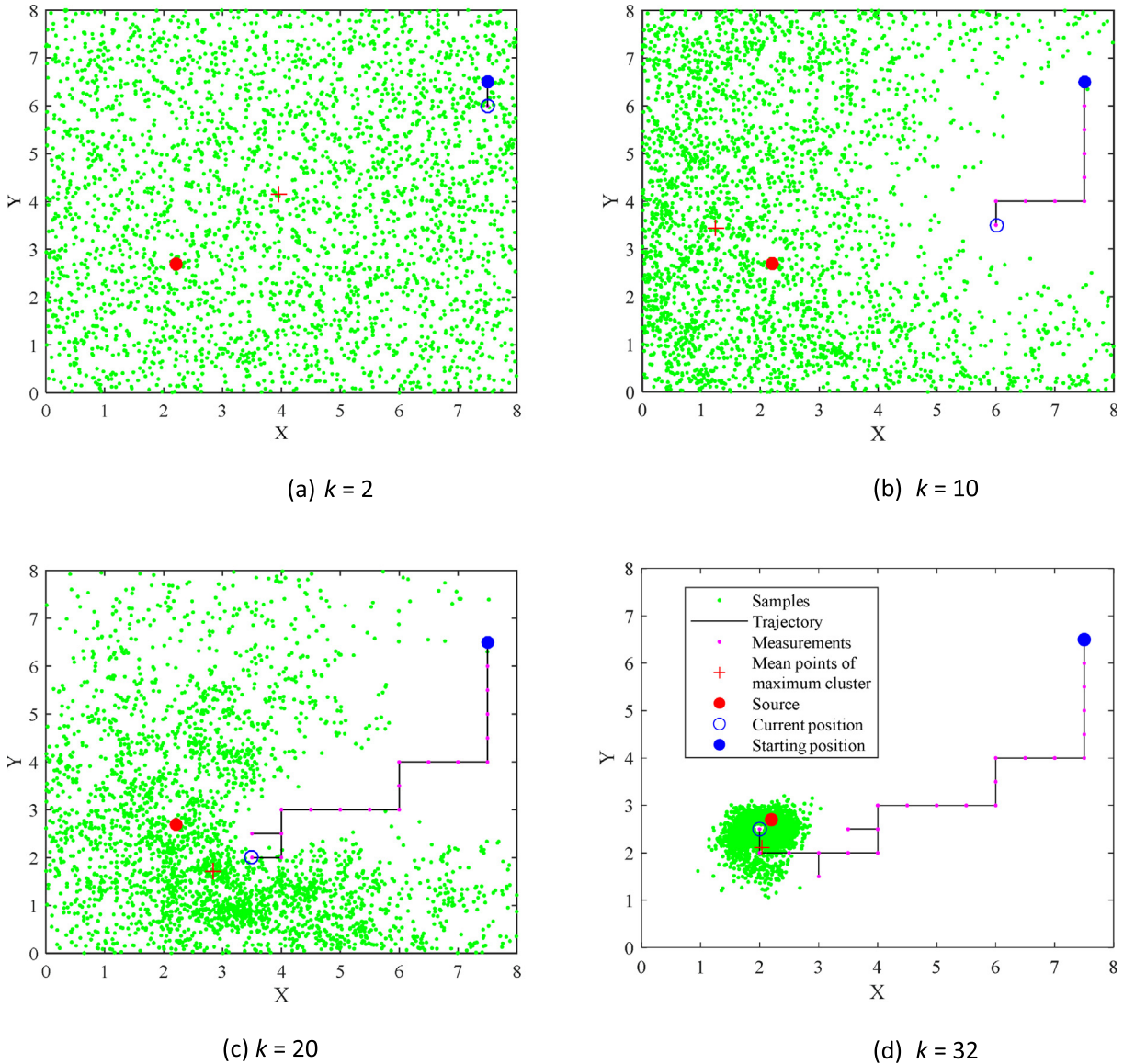


Fig. 7. The typical search process at time step: (a) $k = 2$; (b) $k = 10$; (c) $k = 20$; (d) $k = 32$.

4.3. Discussion on the parameters of the DBSCAN algorithm

The DBSCAN algorithm is utilized to extract the feature of the state, and the clustering results highly depend on the distribution of samples and its parameter (Eps and $MinPts$). Therefore, the determination of parameters is significant. Several works have designed algorithms to adaptively determine parameters [32,33], but they also bring extra computational cost. Especially, in this paper, the clustering process needs to be conducted after each update of the distribution of samples, the extra computational cost is considerable. Therefore, fixed parameters are adopted, and they are selected by extensive experiments.

We conduct a series of experiments for the PC-DQN with different combinations of the Eps and $MinPts$. Each Q-network with different parameters is trained 500 episodes, then they are used to perform the search 100 times in the simulated scenario mentioned in Section 4.1 and the results are shown in Fig. 8. Similar to previous works, the search performance is mainly measured by two indicators, i.e., the success rate (SR) and the mean search steps (MSS). The two indicators evaluate the performance from the aspects of accuracy and efficiency respectively. Note that the algorithm with high SR and low MSS is considered to have good performance.

As shown in Fig. 8, the SR keeps at a high level (>90%) for all parameter combinations, which validates the robustness of our PC-DQN algorithm to the various parameters of the DBSCAN. However, the MSS has a significant fluctuation with the change of parameters. Among these parameter combinations, the PC-DQN performs best (SR achieves 100% and the MSS drops as low as 60.96) when the $Eps = 0.2$ and the $MinPts = 15$. Therefore, the two parameters are chosen for PC-DQN in this simulated scenario.

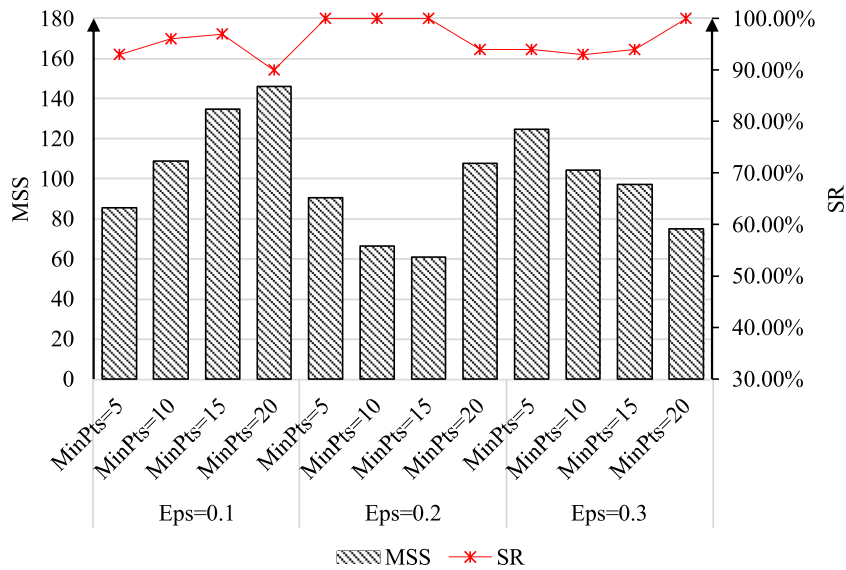


Fig. 8. The search performance of PC-DQN with different parameters of the DBSCAN algorithm.

Table 2

Search performance of three algorithms for different source strengths Q and wind speed V conditions after 500 Monte Carlo simulations.

Q	0.5		1		2	
	0	1	0	1	0	1
RANDOM						
SR	54.6%	53.6%	56.4%	54.8%	58.0%	56.8%
MSS	152.5934	148.9851	145.2589	159.5328	160.3414	145.8768
Entrotaxis						
SR	98.6%	95.0%	99.4%	98.4%	99.8%	99.6%
MSS	76.5659	68.6842	55.1650	49.7012	45.9878	41.3193
PC-DQN						
SR	99.8%	100.0%	99.6%	99.8%	100.0%	100.0%
MSS	100.6292	54.9580	61.0401	36.5691	42.2700	26.0100

4.4. Performance comparison

In this section, the RANDOM and the Entrotaxis algorithm are first introduced as the baseline algorithms. Then, extensive Monte Carlo simulations are conducted for our proposed PC-DQN and two baseline algorithms. Finally, we present the experiment results with a brief analysis and make a comparison of the three algorithms

4.4.1. Baseline algorithms

The RANDOM and Entrotaxis are selected as two baseline algorithms for comparison. The RANDOM strategy is widely used due to its simple principle. Besides, the cognitive strategy is a kind of efficient autonomous source searching method and its superiority has been demonstrated by many previous works [11–13]. The Entrotaxis is an advanced cognitive strategy

Table 3

Searching performance of the retrained Q-network in the scenario (10×8) after 500 experiments.

Q	0.5		1		2	
V	0	1	0	1	0	1
SR	99.6%	99.8%	99.8%	100%	100%	99.6%
MSS	111.0984	55.1543	68.5832	39.3140	50.6280	31.3414

Table 4

Searching performance of the retrained Q-network in the scenario (8×4) after 500 experiments.

Q	0.5		1		2	
V	0	1	0	1	0	1
SR	99.8%	100%	99.8%	100%	100%	100%
MSS	67.7515	47.0280	39.5491	25.8180	29.4780	18.8600

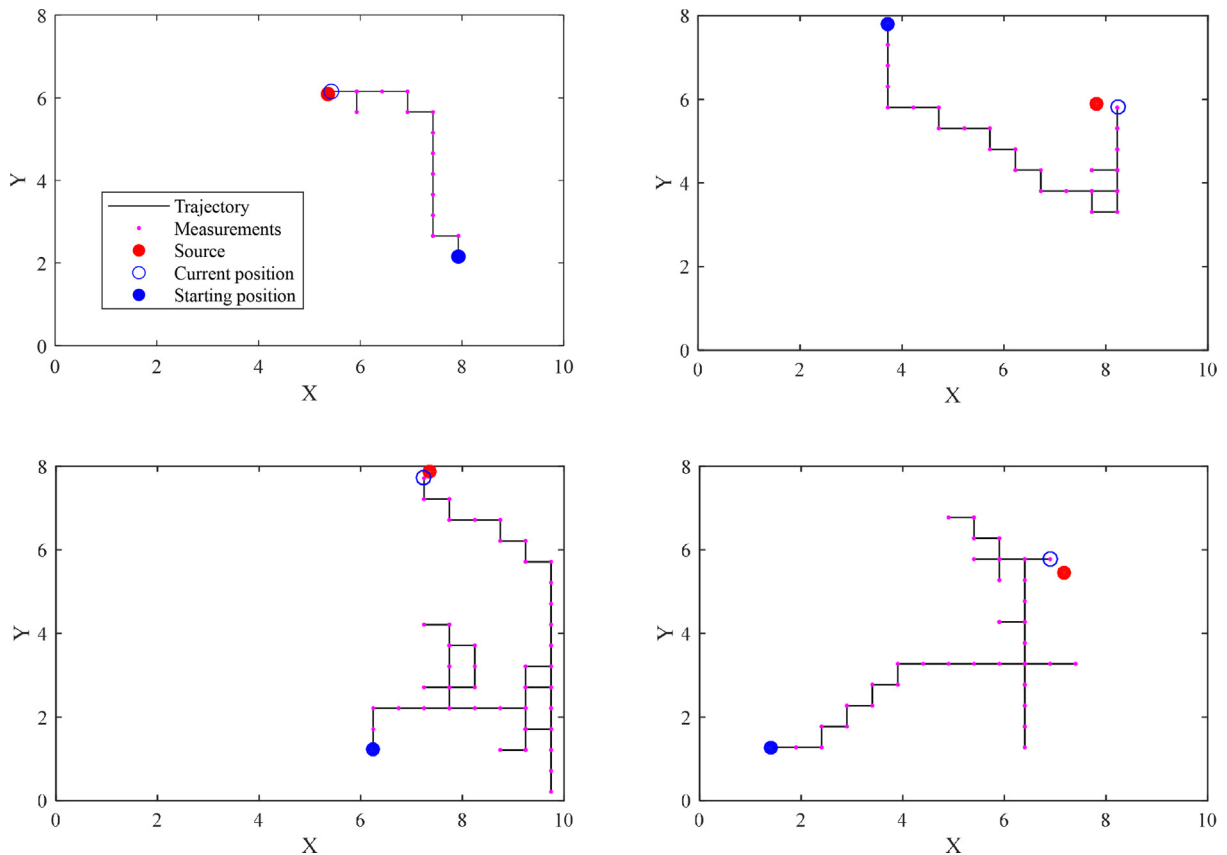


Fig. 9. The searching trajectories of four successful cases in 10×8 scenarios.

that has been demonstrated to outperform other cognitive strategies. Therefore, we choose Entrotaxis as a baseline algorithm.

RANDOM: In each time step of an episode, the searcher randomly chooses an action from the action set, regardless of any other information.

Entrotaxis: The Entrotaxis algorithm [3,12] has been proposed to perform an autonomous search and reconstruction of an emitting source of unknown strength, in turbulent conditions. This approach typically located the source rapidly by using a less computationally demanding reward function without degrading the rate of success. The reward function employed by Entrotaxis refers to the Maximum entropy sampling principles and it considers the entropy of the predictive measurement distribution, given as:

$$I(u_k) \approx - \sum_{\hat{d}_{k+1}}^{\hat{d}^{\max}} P(\hat{d}_{k+1}(\hat{l}_{k+1})) \log P(\hat{d}_{k+1}(\hat{l}_{k+1})) \quad (11)$$

The $P(\hat{d}_{k+1}(\hat{l}_{k+1}))$ can be acquired based on the estimation of source terms, shown as:

$$P(\hat{d}_{k+1}(\hat{l}_{k+1})) \approx \sum_{i=1}^N \frac{(\rho(\hat{l}_{k+1}|p_k^i))^{\hat{d}_{k+1}(\hat{l}_{k+1})}}{\hat{d}_{k+1}(\hat{l}_{k+1})!} e^{-\rho(r_k|p_k^i)} w_k^i \quad (12)$$

4.4.2. Simulation results and analysis

Extensive searching experiments are carried out in the scenario mentioned in Section 4.1. The experiment results of different release strength Q and wind speed V are compared with those of the Entrotaxis algorithm and RANDOM algorithm. Moreover, each group of experiments with different values of Q and V is carried out 500 times.

The experimental results are shown in Table 2. Obviously, the RANDOM algorithm has the worst performance, compared with other algorithms. The SR of the RANDOM keeps at a low level and the MSS exceeds 145 in all scenarios. For both Entrotaxis and PC-DQN, the performances are better in the scenario with strong release strength Q as the MSS decreases with the increase of Q . When Q is set to a certain value, the Entrotaxis has a low MSS in the high wind speed condition but the SR of

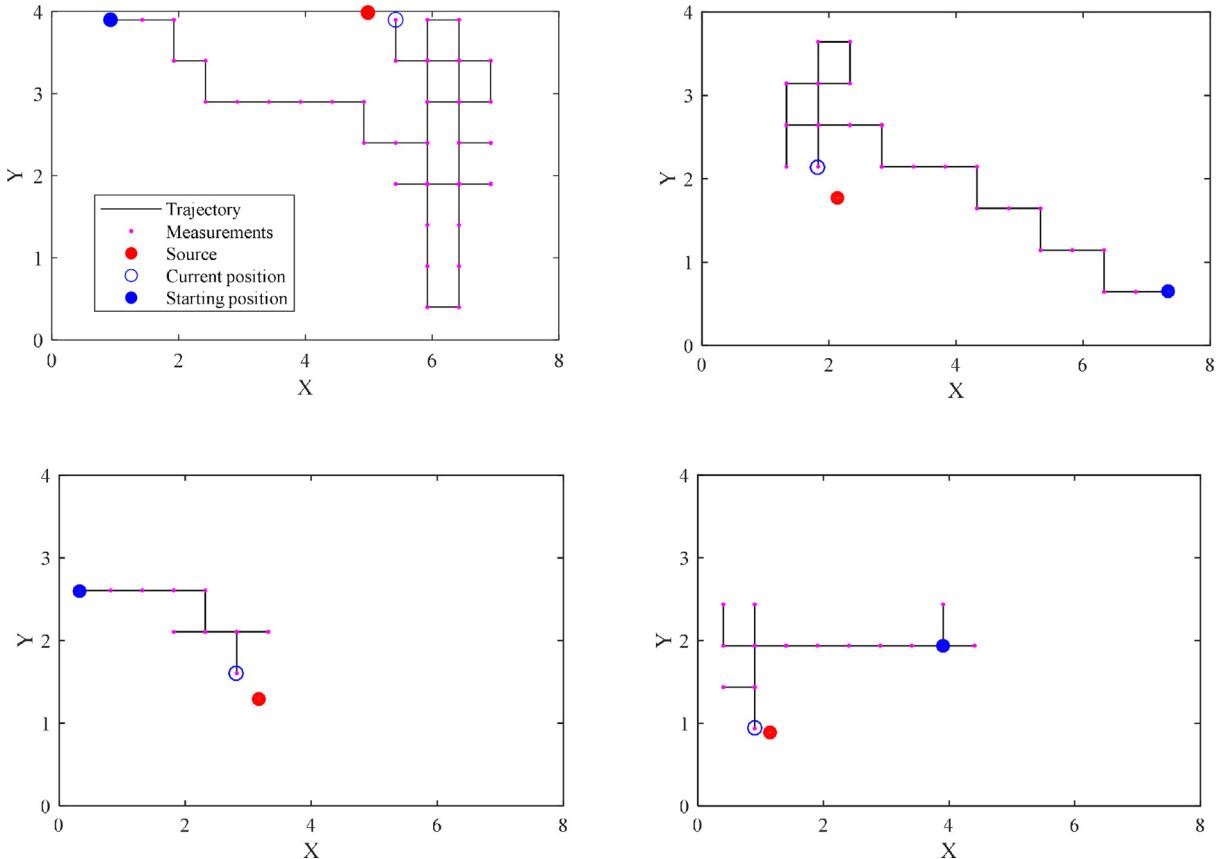


Fig. 10. The searching trajectories of four successful cases in 8×4 scenarios.

Entrotaxis shows a different trend. The SR of the Entrotaxis in the low wind speed condition outperforms the SR in the low wind speed condition, especially when the $Q = 0.5$. In most cases, the PC-DQN algorithm has the highest SR and the lowest MSS compared with RANDOM and Entrotaxis algorithms in the same setting values of Q and V . Specifically, the MSS of the PC-DQN decreases 64.67% than RANDOM and 8.25% than Entrotaxis on average. These results illustrate the most efficient searching ability and best performance of our proposed algorithm.

4.5. Transfer to other scenarios

Obviously, the PC-DQN algorithm mentioned in the previous sections needs a large number of training experiments and training data. However, the trained Q-network cannot maintain good performance in new scenarios. To make better use of the well-trained Q-network and further reduce the number of training experiments and training data, we try to introduce the transfer learning [34,35] technique into our problem.

In general, parameters affecting the dispersion of hazardous gas, e.g. the size of scenarios, source strength, and wind speed, vary in different scenarios. However, as the mechanism of the dispersion of hazardous gas is similar, probably the source searching strategy learned for one scenario can benefit another scenario. With this intuition, we apply the transfer learning concept to retrain the Q-network, which has performed well in a scenario, for the source searching task in another similar scenario.

With the help of the transfer learning method, we retrain the well-trained Q-network in Section 4.1 for source searching in two new scenarios with different sizes. The size of the first scenario is 10×8 , another is 8×4 . In each scenario, the Q-network is retrained with 100 episodes, then it is utilized to perform the source searching task 500 times in the new scenario with different conditions of release strength Q and wind speed V . The results are shown in Table 3 and Table 4.

As shown in Tables 3 and 4, the retrained Q-network also maintains good performance, especially in the strong release strength scenarios, which is consistent with the findings in Section 4.4. The SR of all scenarios are 100% or close to 100%, and the MSS keeps at a low level. Figs. 9 and 10 display the searching trajectories of several successful cases in two scenarios with different sizes. It can be seen that the starting point and the source position are randomly generated in different cases. The existence of these random factors may cause the searcher to take more steps, but the searcher would find the release source finally. The experiment results demonstrate that the well-trained Q-network can be retrained by the transfer learning method for the search missions in new but similar scenarios. Therefore, the demand for the number of training experiments and training data can be effectively reduced and the proposed PC-DQN algorithm will have a generalization performance in various scenarios.

5. Conclusion

This paper leverages the deep reinforcement learning method (PC-DQN) to solve the source searching problem. Since the source searching is a partially observable problem, in which the searcher must make decisions under the uncertainty of the actual source term, we formulate the searching process as the POMDP at first. Then, the POMDP is transformed into the belief MDP to be solved by the deep reinforcement learning approach based on the belief state, which is the estimation of the source term represented by the particle filter. Moreover, the DBSCAN clustering method is applied to extract the feature of the belief state. To deal with the large state space of the source searching problem, we employ a Q-network to replace traditional tables based on the theory of the DQN. A series of simulated experiments of three algorithms (our proposed PC-DQN method and two baselines) are conducted in different simulated scenarios with various environmental parameters. The results demonstrate the good performance of the PC-DQN method proposed in this paper. The DRL method has a long-term view in choosing the action. It utilizes the value function to indicate the long-term desirability of states to achieve the goal. By contrast, the Entrotaxis (baseline) are designed to maximize the informational gain in each step. However, we cannot find a direct relationship between gaining more information and finding the source. Therefore, the DRL-based search method PC-DQN outperforms the Entrotaxis in this paper. With the application of the transfer learning method, we retrain the well-trained Q-network in new scenarios with few data. The experiment results show that the retrained PC-DQN algorithm keeps good performance in new scenarios.

In conclusion, this paper demonstrates the feasibility of the deep reinforcement learning method in solving the source localization problem. However, there are still many works that can be conducted in the future. Firstly, variable step sizes can be utilized and it might be able to improve the search efficiency. Besides, the simulated scenarios in this paper are quite simple. More practical factors in the real application, such as obstacles, terrain restrictions, and dynamic meteorology, should be considered in future works. Correspondingly, the Q-network with a simple structure may be unqualified in the more practical scenario with more complex factors, thus the Q-network with more complex structures may be employed, e.g. Long Short-Term Memory (LSTM) network.

CRedit authorship contribution statement

Yong Zhao: Conceptualization, Methodology, Software. **Bin Chen:** Conceptualization, Methodology. **XiangHan Wang:** Conceptualization, Methodology. **Zhengqiu Zhu:** Conceptualization, Methodology, Formal analysis. **Yiduo Wang:** Data cura-

tion. **Guangquan Cheng**: Writing-review & editing. **Rui Wang**: Validation. **Rongxiao Wang**: Validation. **Ming He**: Validation. **Yu Liu**: Validation.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This study is supported by the National Natural Science Foundation of China under Grant Nos. 62173337, 21808181, 72071207, Humanity and Social Science Youth Foundation of Ministry of Education of China under Grant No. 19YJCZH073, and National Social Science Foundation of China under Grant No. 17CGL047.

References

- [1] L. Yi, N. Shunjiang, W. Wenguo, Development of the public safety system and a security-guaranteed society, *Strategic Study Chin. Acad. Eng.* 19 (1) (2017) 118–123.
- [2] D. Ma et al, Application and improvement of swarm intelligence optimization algorithm in gas emission source identification in atmosphere, *J. Loss Prev. Process Ind.* 56 (2018) 262–271.
- [3] M. Hutchinson, H. Oh, W.-H. Chen, A review of source term estimation methods for atmospheric dispersion events using static or mobile sensors, *Inf. Fusion* 36 (2017) 130–148.
- [4] X.-X. Chen, J. Huang, Odor source localization algorithms on mobile robots: A review and future outlook, *Robot. Autonomous Syst.* 112 (2019) 123–136.
- [5] Q. Lu, Q.-L. Han, S. Liu, A finite-time particle swarm optimization algorithm for odor source localization, *Inf. Sci.* 277 (2014) 111–140.
- [6] P.P. Neumann, V. Hernandez Bennets, A.J. Lilienthal, M. Bartholmai, J.H. Schiller, Gas source localization with a micro-drone using bio-inspired and particle filter-based algorithms, *Adv. Rob.* 27 (9) (2013) 725–738.
- [7] W. Li, J.A. Farrell, S. Pang, R.M. Arrieta, Moth-inspired chemical plume tracing on an autonomous underwater vehicle, *IEEE Trans. Rob.* 22 (2) (2006) 292–307.
- [8] H. Ishida, K.-I. Suetsugu, T. Nakamoto, T. Moriizumi, Study of autonomous mobile sensing system for localization of odor source using gas sensors and anemometric sensors, *Sens. Actuators A: Phys.* 45 (2) (1994) 153–157.
- [9] J.A. Farrell, S. Pang, W. Li, R.M. Arrieta, Biologically inspired chemical plume tracing on an autonomous underwater vehicle, in: 2004 IEEE International Conference on Systems, Man and Cybernetics (IEEE Cat. No. 04CH37583), 2004, vol. 7, IEEE, pp. 5991–5996.
- [10] D.J. Harvey, T.-F. Lu, M.A. Keller, Comparing insect-inspired chemical plume tracking algorithms using a mobile robot, *IEEE Trans. Rob.* 24 (2) (2008) 307–317.
- [11] B. Ristic, A. Skvortsov, A. Gunatilaka, A study of cognitive strategies for an autonomous search, *Inf. Fusion* 28 (2016) 1–9.
- [12] Y. Zhao, B. Chen, Z. Zhu, F. Chen, Y. Wang, D. Ma, Entrotaxis-Jump as a hybrid search algorithm for seeking an unknown emission source in a large-scale area with road network constraint, *Expert Syst. Appl.* 157 (2020) 113484.
- [13] Y. Zhao, B. Chen, Z. Zhu, F. Chen, Y. Wang, Y. Ji, Searching the diffusive source in an unknown obstructed environment by cognitive strategies with forbidden areas, *Build. Environ.* 186 (2020) 107349.
- [14] L.P. Kaelbling, M.L. Littman, A.R. Cassandra, Planning and acting in partially observable stochastic domains, *Artif. Intell.* 101 (1–2) (1998) 99–134.
- [15] B. Ristic, C. Gilliam, W. Moran, J.L. Palmer, Decentralised multi-platform search for a hazardous source in a turbulent flow, *Inf. Fusion* 58 (2020) 13–23.
- [16] M. Hutchinson, C. Liu, W.-H. Chen, Information-based search for an atmospheric release using a mobile robot: Algorithm and experiments, *IEEE Trans. Control Syst. Technol.* 27 (6) (2018) 2388–2402.
- [17] B. Ristic, A. Gunatilaka, Information driven localization of a radiological point source, *Inf. Fusion* 9 (2) (2008) 317–326.
- [18] M. Vergassola, E. Villermaux, B.I. Shraiman, 'Infotaxis' as a strategy for searching without gradients, *Nature* 445 (7126) (2007) 406–409.
- [19] Y. Li, Deep reinforcement learning: An overview, *arXiv preprint arXiv:07274*, 2017..
- [20] J. Choi, K.-E. Kim, Inverse reinforcement learning in partially observable environments, *J. Mach. Learn. Res.* 12 (2011) 691–730.
- [21] P. Zhu, X. Li, P. Poupard, G. Miao, On improving deep reinforcement learning for pomdps, *arXiv preprint arXiv:07978*, 2017..
- [22] E.K. Chong, C.M. Kreucher, A.O. Hero, POMDP approximation using simulation and heuristics, in: *Foundations and applications of sensor management*, Springer, 2008, pp. 95–119..
- [23] K. Azizzadenesheli, A. Lazaric, A. Anandkumar, Reinforcement learning of POMDPs using spectral methods, in: *Conference on Learning Theory*, PMLR, 2016, pp. 193–256..
- [24] H. Kurniawati, D. Hsu, W.S. Lee, Sarsop: efficient point-based pomdp planning by approximating optimally reachable belief spaces, in: *Robotics: Science and systems*, 2008, Citeseer..
- [25] W. Liu, L. Wang, E. Wang, Y. Yang, D. Zeglache, D. Zhang, Reinforcement learning-based cell selection in sparse mobile crowdsensing, *Comput. Netw.* 161 (2019) 102–114.
- [26] R.S. Sutton, A.G. Barto, *Reinforcement learning: An introduction*, MIT Press, 2018.
- [27] J.-L. Wei, Q.-H. Meng, C. Yan, M. Zeng, W. Li, Multi-robot gas-source localization based on reinforcement learning, in: 2012 IEEE International Conference on Robotics and Biomimetics (ROBIO), IEEE, 2012, pp. 1440–1445..
- [28] M. Ester, H.-P. Kriegel, J. Sander, X. Xu, A density-based algorithm for discovering clusters in large spatial databases with noise, *kdd* 96 (34) (1996) 226–231.
- [29] V. Mnih et al, Human-level control through deep reinforcement learning, *Nature* 518 (7540) (2015) 529–533.
- [30] B. Ristic, S. Arulampalam, N. Gordon, *Beyond the Kalman filter: Particle filters for tracking applications*, Artech house, 2003.
- [31] M. Hutchinson, H. Oh, W.-H. Chen, Entrotaxis as a strategy for autonomous search and source reconstruction in turbulent conditions, *Inf. Fusion* 42 (2018) 179–189.
- [32] S. Jahirabadkar, P. Kulkarni, Algorithm to determine ϵ -distance parameter in density based clustering, *Expert Syst. Appl.* 41 (6) (2014) 2939–2946.
- [33] J.-H. Kim, J.-H. Choi, K.-H. Yoo, A. Nasridinov, AA-DBSCAN: an approximate adaptive DBSCAN for finding clusters with varying densities, *J. Supercomput.* 75(1) (2019)..
- [34] S.J. Pan, Q. Yang, A survey on transfer learning, *IEEE Trans. Knowl. Data Eng.* 22 (10) (2009) 1345–1359.
- [35] M.E. Taylor, P. Stone, Transfer learning for reinforcement learning domains: a survey, *J. Mach. Learn. Res.* 10(7) (2009)..